Université Mohamed Premier

École Supérieure de Technologie de Nador

Département d'Innovation Technologique et Ingénierie Informatique

Filière d'Ingénierie Logicielle et Cybursécurité

Rapport sur le theme :

PDF Chatbot : Conception et Développement d'un Agent IA Multimodal pour l'Analyse de Documents PDF

Réalisé par :

Mme. ESSADIKI DOUAE

Mme. AIT ADDI THITRITE

Encadré par :

M. ESBAI REDOUANE

Professeur à l'ESTN

Année universitaire : 2025/2026

Table des matières

Introduction Générale **2**

1 Architecture Technique **3**

1.1 Vue d'Ensemble 3

1.2 Structure du Projet 3

1.2.1 Module Backend (Spring Boot — Java 21) 3

1.2.2 Module Frontend (Angular 17 — TypeScript) 3

1.3 Technologies Utilisées 4

1.3.1 Backend 4

1.3.2 Frontend 4

2 Fonctionnalités de l'Application **5**

2.1 Upload et Gestion des Documents PDF 5

2.2 Interface de Chat Textuel 5

2.3 Interaction Vocale Bidirectionnelle 5

2.3.1 Reconnaissance Vocale — Speech-to-Text (STT) 5

2.3.2 Synthèse Vocale — Text-to-Speech (TTS) 5

2.4 Indicateur de Statut Backend 6

3 Méthodologie de Développement **7**

3.1 Approche Adoptée 7

4 Résultats et Démonstration **8**

4.1 État Final de l'Application 8

4.2 Fonctionnalités Validées 8

4.3 Capture d'Écran de l'Application 8

Conclusion Générale **9**

Introduction Générale

Dans un contexte où la numérisation des documents s'est imposée comme une réalité incontournable dans tous les secteurs d'activité, l'accès rapide et intelligent à l'information contenue dans des fichiers PDF représente un défi majeur pour les professionnels, les étudiants et les chercheurs. Face à ce constat, le présent projet propose le développement d'un agent IA multimodal baptisé **PDF Chatbot**, une application web full-stack permettant à l'utilisateur d'interagir avec ses documents PDF en langage naturel, aussi bien par écrit que par la voix. Reposant sur une architecture moderne combinant un backend Java avec Spring Boot, un frontend Angular 17 et le modèle de langage Llama 3.1 accessible via l'API Groq, cette solution offre une expérience utilisateur fluide intégrant l'upload de documents, la génération de réponses contextualisées et une interface vocale bidirectionnelle. Ce rapport présente successivement l'architecture technique du système, ses fonctionnalités principales, la méthodologie de développement adoptée ainsi que les résultats obtenus, avant de conclure par un bilan et des perspectives d'évolution.

1. Architecture Technique

1.1 Vue d'Ensemble

L'application suit une architecture client-serveur classique, enrichie d'une couche d'intelligence artificielle. Le flux de données entre les composants peut être résumé ainsi :

```
Utilisateur ⟷ Angular 17 ⟷ Spring Boot ⟷ Groq API
                                (Llama 3.1)
```

Lorsqu'un utilisateur uploade un PDF, le backend extrait son texte via Apache PDFBox et le conserve en mémoire associé à un identifiant unique (UUID). Lors d'une question, le backend construit un prompt structuré combinant le texte du document et la question de l'utilisateur, puis interroge l'API Groq avec le modèle `llama-3.1-8b-instant`. Si aucun document n'est chargé, le chatbot répond en mode général. La réponse est retournée au frontend qui l'affiche et peut la lire à voix haute.

1.2 Structure du Projet

Le projet est organisé en deux modules indépendants.

1.2.1 Module Backend (Spring Boot — Java 21)

- **PdfChatbotApplication.java** : Point d'entrée de l'application Spring Boot.
- **CorsConfig.java** : Configuration CORS autorisant les requêtes cross-origin depuis Angular (port 4200).
- **PdfController.java** : Endpoints REST pour l'upload et la liste des documents PDF.
- **ChatController.java** : Endpoints `POST /api/chat/ask` et `GET /api/chat/health`.
- **PdfService.java** : Extraction du texte PDF via Apache PDFBox 3.x (`Loader.loadPDF`).
- **ChatService.java** : Construction du prompt et appel direct à l'API Groq via `java.net.http.Http`

1.2.2 Module Frontend (Angular 17 — TypeScript)

- **app.component.ts** : Layout principal avec indicateur de statut backend.
- **api.service.ts** : Service HTTP pour les appels vers le backend Spring Boot.
- **voice.service.ts** : Service gérant la reconnaissance vocale (Web Speech API) et la synthèse vocale.
- **upload.component.ts** : Composant drag-and-drop pour l'upload de PDFs.
- **chat.component.ts** : Interface de chat avec bulles de messages et contrôles vocaux.

1.3 Technologies Utilisées

1.3.1 Backend

Technologie	**Version**	**Rôle**
Java	21 LTS	Langage principal, inclut `java.net.http.HttpClient`
Spring Boot	3.3.4	Framework backend, gestion REST et dépendances
Apache PDFBox	3.0.2	Extraction du texte depuis les fichiers PDF
Groq API	Llama 3.1-8b	Modèle LLM pour la génération de réponses IA
Maven	3.9+	Gestion des dépendances et build
Lombok	Latest	Réduction du code boilerplate

TABLE 1.1 – Technologies utilisées côté backend

1.3.2 Frontend

Technologie	**Version**	**Rôle**
Angular	17	Framework SPA (Single Page Application)
TypeScript	5.4	Langage de programmation typé
Angular Material	17	Bibliothèque de composants UI (thème violet)
Web Speech API	Native	STT (parole vers texte) et TTS (texte vers parole)
RxJS	7.8	Programmation réactive et gestion des flux HTTP

TABLE 1.2 – Technologies utilisées côté frontend

2. Fonctionnalités de l'Application

2.1 Upload et Gestion des Documents PDF

L'interface d'upload propose une zone drag-and-drop intuitive acceptant les fichiers PDF jusqu'à 50 Mo. Une fois le fichier déposé, le backend extrait automatiquement le texte via Apache PDFBox et génère un UUID unique pour référencer le document. La liste des documents disponibles est affichée dans la sidebar gauche, permettant à l'utilisateur de basculer entre plusieurs documents.

2.2 Interface de Chat Textuel

La fenêtre de chat affiche l'historique des échanges sous forme de bulles colorées : violet foncé pour les messages de l'utilisateur, violet clair pour les réponses de l'assistant. La barre de saisie reste toujours visible, même en l'absence de document, affichant alors un avertissement invitant l'utilisateur à charger un PDF. Le chatbot peut également répondre à des questions générales sans document chargé.

2.3 Interaction Vocale Bidirectionnelle

Le module vocal constitue l'une des caractéristiques distinctives de cette application. Il repose sur la Web Speech API intégrée aux navigateurs modernes et propose deux fonctionnalités complémentaires.

2.3.1 Reconnaissance Vocale — Speech-to-Text (STT)

L'utilisateur peut activer l'enregistrement vocal en cliquant sur le bouton microphone. La parole est transcrite en temps réel dans la barre de saisie et envoyée automatiquement à la fin de la prise de parole. Cette fonctionnalité est optimisée pour Google Chrome et Microsoft Edge.

2.3.2 Synthèse Vocale — Text-to-Speech (TTS)

Chaque réponse de l'assistant est accompagnée d'un bouton permettant de la lire à voix haute. Un mode ≪ Voix auto ≫ peut être activé pour que toutes les réponses soient lues automatiquement sans intervention manuelle. La synthèse vocale utilise les voix système du navigateur, disponibles sur tous les navigateurs modernes.

2.4 Indicateur de Statut Backend

La toolbar principale affiche en temps réel le statut de connexion au backend : un badge vert ≪ Backend connecté ≫ lorsque l'API répond, rouge ≪ Serveur inaccessible ≫ dans le cas contraire. Cette vérification est effectuée en interrogeant l'endpoint `GET /api/chat/health` au démarrage du frontend.

3. Méthodologie de Développement

3.1 Approche Adoptée

Le développement a suivi une approche itérative et incrémentale. Face aux nombreux obstacles techniques rencontrés, chaque problème a été isolé, diagnostiqué et résolu de manière systématique avant de passer au suivant. Cette méthode a permis de maintenir une base de code stable tout en avançant progressivement vers les objectifs fixés, en testant chaque fonctionnalité indépendamment avant de les intégrer dans l'ensemble du système.

4. Résultats et Démonstration

4.1 État Final de l'Application

À l'issue du développement, l'application PDF Chatbot est pleinement fonctionnelle. Le backend Spring Boot démarre correctement sur le port 8080 et expose les endpoints REST nécessaires. Le frontend Angular tourne sur le port 4200 et communique avec le backend via les appels HTTP configurés dans le service `api.service.ts`.

4.2 Fonctionnalités Validées

- Upload de fichiers PDF par glisser-déposer ou sélection, avec extraction automatique du texte.
- Affichage de la liste des documents uploadés avec indication du document actif.
- Interface de chat avec barre de saisie toujours visible et historique scrollable.
- Génération de réponses par Groq API (Llama 3.1) basées sur le contenu du PDF ou en mode général.
- Reconnaissance vocale activée par le bouton microphone, transcription automatique.
- Synthèse vocale des réponses (lecture manuelle ou mode automatique).
- Indicateur de statut backend (vert/rouge) dans la toolbar.
- Design cohérent avec thème violet (Angular Material, couleurs personnalisées).

4.3 Capture d'Écran de l'Application

La figure ci-dessous illustre l'application en cours d'utilisation : un CV PDF a été uploadé, et l'utilisateur a posé la question ≪donner le parcours de Douae≫ via le chat. Le modèle Llama 3.1 (via Groq API) a analysé le document et fourni une réponse structurée détaillant le parcours académique et professionnel de la personne.

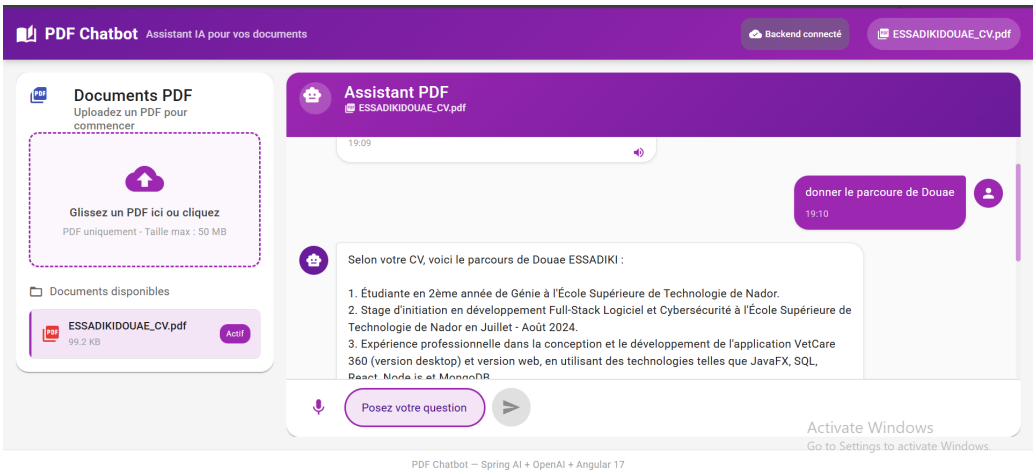

FIGURE 4.1 – Interface du PDF Chatbot — analyse d'un CV en temps réel

Conclusion Générale

Ce projet a permis de concevoir et de développer une application web complète intégrant plusieurs technologies modernes : Spring Boot pour le backend, Angular 17 pour le frontend, Apache PDFBox pour l'extraction de texte, et le modèle Llama 3.1 via l'API Groq comme moteur d'intelligence artificielle. L'ensemble forme un agent IA cohérent et fonctionnel, capable d'analyser des documents PDF et d'interagir avec l'utilisateur de manière naturelle, aussi bien par écrit que par la voix. Les compétences développées au cours de ce projet couvrent un spectre large allant du développement full-stack à l'intégration de modèles de langage, en passant par la gestion des APIs REST, les fonctionnalités vocales natives des navigateurs et la résolution de problèmes d'incompatibilité de dépendances. Plusieurs axes d'amélioration sont envisageables pour la suite : l'intégration d'un vector store pour améliorer la précision sur les longs documents, l'ajout d'un moteur OCR pour traiter les PDFs scannés, la persistance des données via une base de données, et un déploiement cloud en production. Ce projet constitue ainsi une base solide, extensible et réutilisable pour des développements futurs dans le domaine de l'analyse documentaire intelligente.

Lien GitHub :

`https://github.com/tithrite/aiAgent.git`